

Relatório de Implementação - Rede Neural Quantizada em Assembly RISC-V

Visão Geral da Estrutura de Memória:

A memória foi dividida em blocos principais:

- `.data`: caractere de nova linha (barN).
- `.bss`: buffers para entrada, vetores, tamanhos e cópias temporárias.

Cada vetor possui um tamanho máximo de 6000 bytes para garantir espaço suficiente mesmo para redes grandes.

Descrição das Funções:

- `_start`

Ponto de entrada do programa. Organiza a execução em 7 etapas principais:

1. Leitura da entrada (3 linhas)
2. Processamento da arquitetura
3. Extração do vetor de entrada
4. Execução da inferência camada a camada
5. Cálculo do argmax na saída
6. Impressão da classe prevista
7. Encerramento do programa

- `gets`

Realiza a leitura de 3 linhas da entrada padrão, salvando os dados em entrada.

- `interpretar_numeros`

Converte uma string com números separados por vírgula em inteiros, armazenando-os sequencialmente em um vetor e informando a quantidade total lida.

- `parse_numeros`

Responsável por extrair um número por vez da string, até encontrar um separador (',' ou ']') e salvá-lo como string em cópia.

- `atoi`

Converte uma string numérica em inteiro decimal. Trata espaços e sinais negativos.

- *itoa*

Converte um inteiro para string. Usa base 10 por padrão e realiza inversão do buffer ao final.

- *puts*

Imprime uma string caractere por caractere, finalizando com uma nova linha (\n).

- *exit*

Finaliza o programa com código de saída 0 (sucesso).

- *contador_fim*

Avança o ponteiro até encontrar '}' para ignorar os blocos Ln e chegar ao vetor de entrada.

- *scaneamento*

Localiza o próximo bloco de pesos ("Ln") dentro da string e posiciona o ponteiro no início dos números do vetor.

- *executar_inferencia*

Realiza a execução forward da rede:

- Lê os pesos de cada Ln
- Multiplica pela entrada atual
- Aplica a função de ativação ReLU
- Atualiza a matriz de entrada com os novos valores

- *multiplica_matriz*

Realiza o produto escalar entre cada vetor Ln e o vetor de entrada. Salva os resultados em matrizCopia.

- *troca*

Copia os valores de matrizCopia para matriz, atualizando o tamanho para tamanho3.

- *relu*

Aplica a função ReLU a cada valor do vetor: se for negativo, torna zero.

- *argmax*

Percorre um vetor e retorna o índice do maior valor encontrado.

Justificativas:

- Modularização: Cada tarefa foi isolada em uma função específica para facilitar a manutenção e depuração.
- Manipulação de Strings: Criada para contornar a limitação do Assembly quanto a parsing.
- Uso de pilha: Preservamos o estado entre chamadas usando a pilha para evitar corromper registros importantes.
- Generalização: A rede aceita qualquer número de camadas e qualquer dimensão (limitado apenas pela memória).